Салимли Айзек Мухтар оглы **гр. 5140201/60301**

ИДЗ №2 (Цепи Маркова)

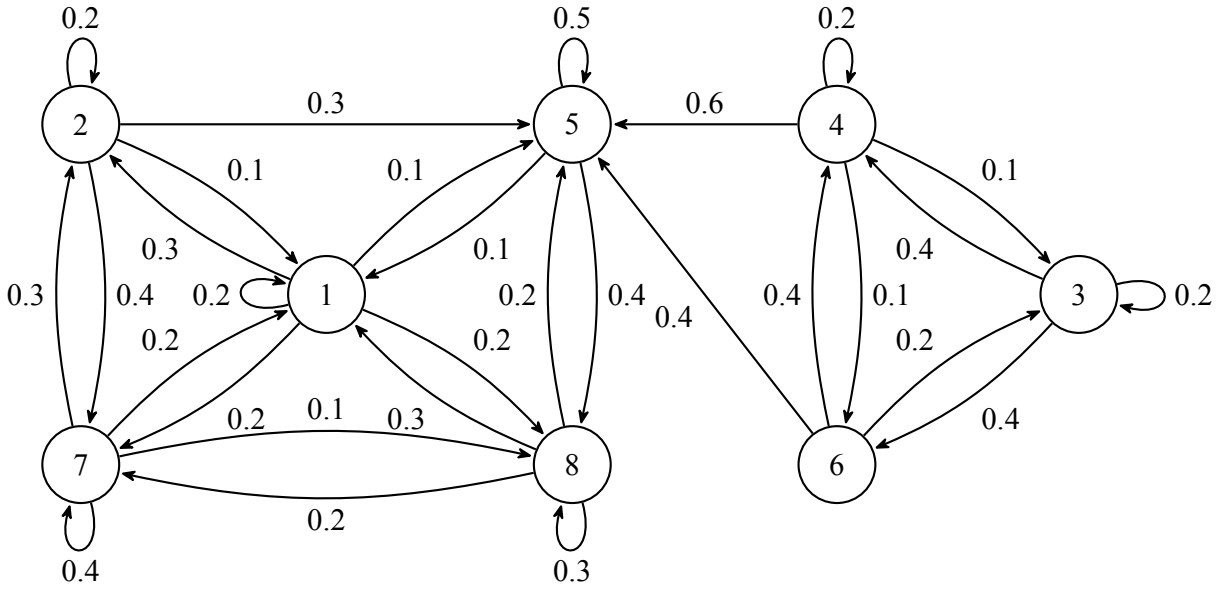
Вариант 16

Задание:

Вар. 16 (30126)
Матрица вероятностей перехода однородной цепи Маркова имеет вид

$$\frac{1}{10}\begin{pmatrix} 2 & 3 & 0 & 0 & 1 & 0 & 2 & 2 \\ 1 & 2 & 0 & 0 & 3 & 0 & 4 & 0 \\ 0 & 0 & 2 & 4 & 0 & 4 & 0 & 0 \\ 0 & 0 & 1 & 2 & 6 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 5 & 0 & 0 & 4 \\ 0 & 0 & 2 & 4 & 4 & 0 & 0 & 0 \\ 2 & 3 & 0 & 0 & 0 & 0 & 4 & 1 \\ 3 & 0 & 0 & 0 & 2 & 0 & 2 & 3 \end{pmatrix}.$$

1. Определить матрицу вероятностей перехода за два шага.
2. Выделить классы сообщающихся состояний.
3. Есть ли невозвратные состояния?
4. Найти период в каждом из классов.
5. Вычислить финальные вероятности в каждом классе.
6. Смоделировать траектории цепи Маркова длины 10, 50, 100 и 1000 шагов, начинающиеся в различных состояниях для каждого случая.
7. Вычислить процент времени нахождения ЦМ в каждом из состояний. Сравнить результат с вектором финальных вероятностей.



№ 1:

$$\mathbb{P}^{(2)} = \mathbb{P}^2 = \frac{1}{100}\begin{pmatrix} 18 & 18 & 0 & 0 & 20 & 0 & 28 & 16 \\ 15 & 19 & 0 & 0 & 22 & 0 & 26 & 18 \\ 0 & 0 & 16 & 32 & 40 & 12 & 0 & 0 \\ 6 & 0 & 6 & 12 & 46 & 6 & 0 & 24 \\ 19 & 3 & 0 & 0 & 34 & 0 & 10 & 34 \\ 4 & 0 & 8 & 16 & 44 & 12 & 0 & 16 \\ 18 & 24 & 0 & 0 & 13 & 0 & 34 & 11 \\ 21 & 15 & 0 & 0 & 19 & 0 & 20 & 25 \end{pmatrix}$$

$18/100 + 18/100 + 0 + 0 + 20/100 + 0 + 28/100 + 16/100 = 1$ - верно

$15/100 + 19/100 + 0 + 0 + 22/100 + 0 + 26/100 + 18/100 = 1$ - верно

$0 + 0 + 16/100 + 32/100 + 40/100 + 12/100 + 0 + 0 = 1$ - верно

$6/100 + 0 + 6/100 + 12/100 + 46/100 + 6/100 + 0 + 24/100 = 1$ - верно

$19/100 + 3/100 + 0 + 0 + 34/100 + 0 + 10/100 + 34/100 = 1$ - верно

$4/100 + 0 + 8/100 + 16/100 + 44/100 + 12/100 + 0 + 16/100 = 1$ - верно

$18/100 + 24/100 + 0 + 0 + 13/100 + 0 + 34/100 + 11/100 = 1$ - верно

$21/100 + 15/100 + 0 + 0 + 19/100 + 0 + 20/100 + 25/100 = 1$ - верно

№ 2:

Цепь состоит из 2 классов сообщающихся состояний: $\mathbf{C}_1 = \{1, 2, 5, 7, 8\}$ и $\mathbf{C}_2 = \{3, 4, 6\}$.

$\mathbf{C}_1$	1	2	5	7	8
1	$1 \to 1$	$1 \to 2$	$1 \to 5$	$1 \to 7$	$1 \to 8$
2	$2 \to 1$	$2 \to 2$	$2 \to 5$	$2 \to 7$	$2 \to 1 \to 8$
5	$5 \to 1$	$5 \to 1 \to 2$	$5 \to 5$	$5 \to 8 \to 7$	$5 \to 8$
7	$7 \to 1$	$7 \to 2$	$7 \to 1 \to 5$	$7 \to 7$	$7 \to 8$
8	$8 \to 1$	$8 \to 1 \to 2$	$8 \to 5$	$8 \to 7$	$8 \to 8$

$\mathbf{C}_2$	3	4	6
3	$3 \to 3$	$3 \to 4$	$3 \to 6$
4	$4 \to 3$	$4 \to 4$	$4 \to 6$
6	$6 \to 3$	$6 \to 4$	$6 \to 3 \to 6$

Класс $\mathbf{C}_1$ замкнутый. Т.е из 1, 2, 5, 7, 8 стрелки ведут только в 1, 2, 5, 7, 8. А класс $\mathbf{C}_2$ незамкнутый, т.к. из него есть переходы в 5 ($4 \to 5$, $p_{45} = 0.6$; $6 \to 5$, $p_{65} = 0.4$).

Вершины 3, 4, 6 - недостижимые из вершин класса $\mathbf{C}_1$ (т.к. $\exists$ пути из них в $\mathbf{C}_1$, но из $\mathbf{C}_1$ в них нет), а значит вершины 3, 4, 6 несущественные.

№ 3:

Так как несущественные состояния - невозвратные то и вершины 3, 4, 6 невозвратные.

№ 4:

Период класса $T = \text{НОД}(T_n)$, $T_n = \{n \geq 1 : p_{ii}^{(n)} > 0\}$ - длины всех возвратов в состояние i класса. У состояний 1, 2, 3, 4, 5, 7, 8 есть петля, поэтому $p_{ii}^{(n)} \geq (p_{ii})^n > 0$ при любом n. У состояния 6 петли нет но есть возвраты ($6 \to 3 \to 6$, $6 \to 3 \to 3 \to 6$, ...):

$$\mathbf{C}_1 = \{1, 2, 5, 7, 8\}: \quad T_n = \{1, 2, 3, \ldots\}, \quad T = \text{НОД}(1, 2, 3, \ldots) = 1$$

$$\mathbf{C}_2 = \{3, 4, 6\}: \quad T_n = \{1, 2, 3, \ldots\}\ (i = 3, 4), \quad T_n = \{2, 3, 4, \ldots\}\ (i = 6),$$

$$T = \text{НОД}(1, 2, 3, \ldots) = 1$$

То есть все классы непериодичны.

№ 5:

Вектор финальных вероятностей класса $\mathbf{C}_i$: $\pi_{\mathbf{C}_i} = \pi_{\mathbf{C}_i} \cdot \mathbb{P}_{\mathbf{C}_i}$.

$$\pi_{\mathbf{C}_1} = (\pi_1, \pi_2, \pi_5, \pi_7, \pi_8), \qquad \mathbb{P}_{\mathbf{C}_1} = \frac{1}{10}\begin{pmatrix} 2 & 3 & 1 & 2 & 2 \\ 1 & 2 & 3 & 4 & 0 \\ 1 & 0 & 5 & 0 & 4 \\ 2 & 3 & 0 & 4 & 1 \\ 3 & 0 & 2 & 2 & 3 \end{pmatrix}$$

$$\begin{cases} 10\pi_1 = 2\pi_1 + \pi_2 + \pi_5 + 2\pi_7 + 3\pi_8 \\ 10\pi_2 = 3\pi_1 + 2\pi_2 + 3\pi_7 \\ 10\pi_5 = \pi_1 + 3\pi_2 + 5\pi_5 + 2\pi_8 \\ 10\pi_7 = 2\pi_1 + 4\pi_2 + 4\pi_7 + 2\pi_8 \\ 10\pi_8 = 2\pi_1 + 4\pi_5 + \pi_7 + 3\pi_8 \\ 1 = \pi_1 + \pi_2 + \pi_5 + \pi_7 + \pi_8 \end{cases} \Rightarrow \begin{cases} \pi_1 = \dfrac{410}{2232} \approx 0.1837 \\ \pi_2 = \dfrac{351}{2232} \approx 0.1573 \\ \pi_5 = \dfrac{479}{2232} \approx 0.2146 \\ \pi_7 = \dfrac{526}{2232} \approx 0.2357 \\ \pi_8 = \dfrac{466}{2232} \approx 0.2088 \end{cases}$$

Для незамкнутого класса $\mathbf{C}_2 = \{3, 4, 6\}$: $\pi_3 = \pi_4 = \pi_6 = 0$, так как состояния 3, 4, 6 невозвратные.

№ 6:

Поиск траектории был реализован на языке python. Для разных классов сообщающихся состояний для различных вершин были рассмотрены траектории длины 10, 50, 100, 1000.

Функция simulate_markov_chain моделирует траекторию цепи - на каждом шаге следующее состояние выбирается случайно с вероятностями из строки матрицы переходов для текущего состояния.

```
def simulate_markov_chain(transition_matrix, initial_state, steps):
    states = [initial_state]
    current_state = initial_state

    for _ in range(steps):
        current_state = np.random.choice(
            a=len(transition_matrix),
            p=transition_matrix[current_state]
        )
        states.append(current_state)
```

```
    return states
```

Листинг 1: Функция моделирования траектории цепи Маркова

Результаты (Initial State - начальное состояние, Steps - число шагов, Trajectory - полученная траектория, то есть последовательность состояний цепи):

Initial State: 1, Steps: 10, Trajectory: [1, 2, 5, 5, 1, 2, 2, 7, 1, 1, 8]

Initial State: 3, Steps: 50, Trajectory: [3, 4, 3, 6, 3, 4, 4, 5, 5, 5, 5, 5, 8, 7, 2, 5, 1, 7, 7, 7, 7, 1, 7, 1, 7, 2, 5, 8, 7, 2, 1, 7, 7, 1, 5, 5, 5, 5, 5, 8, 1, 2, 1, 7, 2, 7, 7, 2, 7, 8, 8]

Initial State: 5, Steps: 100, Trajectory: [5, 5, 8, 1, 7, 7, 7, 7, 7, 7, 7, 1, 1, 7, 2, 5, 5, 5, 5, 5, 8, 8, 8, 5, 5, 5, 8, 8, 8, 1, 7, 8, 1, 8, 5, 8, 1, 8, 7, 8, 8, 5, 8, 8, 1, 1, 7, 7, 8, 7, 1, 2, 5, 5, 5, 1, 8, 8, 5, 5, 8, 7, 7, 7, 7, 1, 8, 1, 1, 8, 7, 7, 7, 7, 7, 1, 7, 2, 7, 2, 7, 2, 7, 1, 1, 7, 7, 1, 5, 5, 5, 5, 1, 8, 1, 8, 8, 1, 5, 8, 1]

Initial State: 7, Steps: 1000, Trajectory: [7, 7, 7, 2, 2, 5, 5, 8, 7, 2, 2, 5, 5, 8, 8, 1, 7, 2, 7, 1, 7, 7, 1, 2, 2, 7, 1, 2, 7, 7, 1, 1, 2, 7, 1, 7, 2, 7, 7, 8, 5, 5, 8, 7, 7, 7, 2, 7, 7, 7, 1, 1, 2, 7, 7, 2, 7, 1, 2, 2, 5, 8, 7, 1, 1, 1, 1, 1, 2, 7, 7, 1, 5, 8, 5, 5, 5, 5, 8, 1, 5, 1, 1, 2, 5, 1, 2, 7, 1, 8, 8, 8, 8, 8, 7, 1, 7, 2, 7, 7, 7, 1, 2, 5, 8, 8, 5, 5, 1, 1, 7, 2, 1, 1, 1, 2, 5, 8, 8, 5, 1, 1, 1, 5, 5, 8, 1, 7, 2, 2, 7, 7, 1, 2, 1, 8, 8, 7, 8, 5, 5, 5, 8, 1, 2, 2, 1, 7, 1, 1, 2, 2, 7, 8, 1, 2, 2, 5, 1, 2, 7, 8, 7, 7, 7, 2, 7, 2, 7, 7, 7, 1, 2, 5, 5, 8, 1, 2, 2, 2, 1, 2, 7, 7, 2, 5, 8, 8, 8, 7, 2, 5, 8, 7, 7, 1, 8, 7, 2, 1, 7, 2, 2, 7, 7, 2, 7, 7, 2, 1, 5, 5, 8, 1, 2, 5, 8, 7, 2, 2, 7, 2, 7, 1, 2, 5, 8, 7, 8, 1, 5, 5, 5, 5, 5, 5, 8, 1, 7, 7, 7, 1, 7, 2, 7, 2, 7, 7, 2, 5, 5, 1, 1, 2, 2, 2, 5, 8, 5, 5, 8, 8, 5, 8, 1, 2, 7, 2, 1, 1, 7, 2, 5, 8, 7, 2, 1, 2, 7, 7, 7, 8, 8, 8, 5, 5, 8, 5, 8, 5, 5, 8, 8, 1, 2, 7, 2, 5, 8, 8, 1, 2, 2, 7, 2, 2, 7, 7, 1, 1, 8, 8, 5, 5, 5, 8, 5, 8, 7, 7, 2, 1, 1, 7, 2, 2, 5, 8, 5, 8, 1, 8, 1, 7, 1, 7, 2, 7, 8, 8, 8, 5, 8, 1, 2, 2, 7, 8, 1, 5, 5, 5, 5, 5, 5, 5, 8, 8, 8, 8, 7, 2, 7, 7, 2, 7, 2, 2, 1, 2, 7, 1, 5, 5, 8, 5, 5, 5, 5, 5, 1, 8, 5, 5, 8, 5, 8, 1, 8, 1, 1, 2, 7, 2, 2, 7, 1, 2, 1, 5, 5, 1, 2, 7, 7, 2, 7, 7, 2, 7, 7, 7, 7, 2, 7, 2, 2, 5, 5, 5, 5, 1, 2, 5, 8, 8, 8, 8, 1, 8, 8, 8, 7, 7, 8, 1, 1, 8, 5, 5, 8, 8, 7, 1, 1, 5, 5, 8, 7, 1, 5, 5, 1, 5, 5, 8, 1, 2, 2, 5, 8, 5, 5, 8, 5, 8, 8, 1, 2, 5, 5, 5, 1, 2, 1, 7, 8, 1, 1, 5, 8, 1, 2, 2, 5, 5, 8, 8, 8, 1, 1, 7, 1, 2, 5, 5, 8, 1, 8, 1, 7, 1, 8, 1, 2, 7, 8, 1, 2, 2, 7, 1, 1, 5, 8, 1, 5, 8, 1, 1, 2, 7, 2, 7, 2, 7, 2, 7, 7, 2, 2, 5, 5, 8, 5, 8, 7, 8, 1, 2, 7, 7, 2, 5, 5, 5, 5, 5, 8, 8, 8, 1, 8, 8, 5, 8, 7, 7, 2, 7, 1, 5, 8, 8, 5, 1, 2, 5, 5, 8, 1, 7, 7, 7, 8, 1, 1, 5, 5, 5, 8, 7, 1, 1, 2, 7, 7, 2, 5, 8, 7, 2, 5, 8, 8, 1, 2, 5, 5, 8, 8, 8, 1, 7, 2, 7, 7, 2, 5, 1, 5, 8, 5, 1, 2, 5, 5, 5, 5, 5, 8, 8, 1, 8, 8, 8, 5, 5, 8, 5, 5, 1, 5, 5, 1, 5, 1, 2, 7, 7, 2, 1, 7, 2, 1, 1, 2, 7, 2, 5, 1, 2, 5, 1, 1, 7, 1, 5, 5, 8, 5, 5, 8, 5, 8, 7, 7, 7, 1, 7, 8, 1, 1, 7, 1, 2, 7, 2, 5, 8, 5, 8, 1, 2, 7, 2, 7, 7, 7, 7, 7, 1, 7, 7, 7, 7, 7, 7, 7, 7, 1, 1, 1, 8, 8, 7, 2, 7, 7, 1, 1, 7, 7, 7, 7, 2, 7, 8, 8, 1, 1, 2, 2, 5, 5, 8, 1, 8, 5, 5, 8, 7, 2, 2, 7, 7, 2, 7, 2, 7, 1, 2, 2, 7, 7, 2, 5, 8, 8, 1, 1, 1, 1, 2, 5, 8, 8, 5, 8, 1, 2, 5, 8, 7, 7, 8, 1, 8, 1, 7, 7, 2, 7, 7, 7, 7, 1, 8, 8, 8, 7, 1, 2, 7, 1, 8, 8, 5, 8, 7, 2, 2, 2, 2, 7, 2, 7, 2, 7, 7, 1, 2, 7, 7, 7, 7, 7, 7, 7, 1, 1, 2, 7, 1, 8, 8, 5, 8, 8, 8, 1, 2, 7, 2, 1, 7, 2, 2, 5, 5, 5, 8, 1, 8, 1, 2, 7, 2, 5, 5, 5, 5, 5, 8, 5, 5, 8, 8, 7, 2, 5, 5, 5, 5, 1, 2, 7, 2, 7, 7, 2, 5, 5, 5, 8, 1, 1, 8, 1, 7, 7, 7, 7, 2, 5, 8, 5, 5, 8, 5, 1, 7, 7, 8, 1, 2, 5, 8, 5, 1, 7, 7, 1, 2, 7, 2, 2, 7, 2, 5, 5, 8, 8, 7, 2, 2, 1, 8, 8, 8, 7, 2, 7, 2, 2, 5, 5, 5, 5, 1, 1, 2, 5, 5, 8, 1, 2, 7, 2, 2, 5, 8, 1, 8, 7, 2, 5, 5, 8, 7, 8, 8, 1, 1, 2, 7, 1, 8, 1, 8, 5, 8, 8, 8, 8, 8, 5, 8, 1, 2, 7, 7, 1, 1, 7, 2, 2, 7, 1, 7, 8, 8, 8, 7, 1, 2, 5, 5, 8, 7, 1, 2, 5, 5, 5, 5, 8, 7, 7, 7, 2, 5, 5, 8, 1, 5, 1, 2, 5, 8, 1, 8, 1, 1, 1]

Initial State: 6, Steps: 100, Trajectory: [6, 5, 5, 5, 8, 1, 1, 7, 7, 7, 2, 7, 7, 2, 5, 5, 5, 8, 5, 5, 5, 8, 7, 7, 2, 7, 7, 2, 2, 5, 5, 8, 7, 7, 1, 7, 7, 2, 7, 7, 7, 1, 1, 1, 2, 7, 7, 2, 7, 2, 1, 7, 2, 5, 8, 7, 7, 7, 7, 1, 2, 7, 1, 2, 7, 2, 5, 8, 5, 5, 5, 5, 5, 8, 1, 2, 7, 2, 5, 5, 5, 5, 8, 1, 7, 7, 1, 8, 8, 8, 8, 8, 8, 1, 2, 7, 7, 7, 1, 7, 8]

Initial State: 8, Steps: 1000, Trajectory: [8, 7, 2, 1, 8, 5, 8, 7, 2, 7, 2, 7, 7, 2, 7, 7, 7, 2, 1, 7, 7, 7, 7, 7, 1, 2, 5, 8, 8, 1, 5, 5, 5, 5, 8, 5, 5, 8, 5, 5, 5, 5, 1, 7, 1, 5, 5, 8, 7, 7, 1, 8, 5, 8, 8, 1, 1, 2, 2, 5, 5, 8, 7, 1, 5, 5, 5, 1, 5, 5, 8, 7, 7, 2, 7, 2, 2, 5, 5, 8, 1, 1, 1, 8, 1, 7, 2, 7, 7, 7, 1, 2, 7, 1, 7, 1, 7, 7, 7, 2, 7, 8, 7, 1, 7, 7, 2, 7, 2, 5, 5, 5, 5, 5, 8, 1, 1, 1, 5, 5, 5, 8, 5, 5, 5, 8, 8, 8, 7, 8, 8, 5, 8, 7, 2, 7, 1, 8, 8, 8, 1, 1, 2, 7, 7, 2, 2, 7, 7, 1, 8, 8, 8, 1, 7, 8, 5, 5, 5, 8, 1, 2, 1, 2, 5, 5, 1, 2, 7, 1, 7, 2, 5, 8, 7, 1, 8, 1, 2, 7, 2, 7, 7, 2, 7, 1, 1, 8, 1, 8, 1, 7, 7, 7, 7, 7, 7, 7, 8, 8, 5, 5, 5, 5, 5, 5, 5, 8, 1, 7, 7, 1, 7, 1, 8, 1, 2, 2, 7, 8, 8, 5, 8, 8, 7, 7, 1, 8, 5, 8, 8, 5, 8, 5, 5, 8, 5, 1, 7, 7, 7, 8, 8, 1, 2, 5, 5, 8, 1, 5, 5, 8, 7, 7, 7, 2, 5, 5, 8, 1, 1, 1, 1, 7, 7, 1, 7, 8, 7, 2, 7, 7, 7, 8, 1, 1, 2, 7, 2, 5, 8, 1, 2, 1, 2, 7, 1, 8, 8, 7, 2, 5, 5, 8, 8, 5, 5, 1, 1, 2, 2, 7, 2, 1, 8, 1, 7, 8, 7, 2, 2, 7, 2, 5, 5, 5, 5, 1, 7, 7, 2, 5, 5, 8, 1, 7, 7, 2, 5, 5, 5, 8, 8, 7, 7, 1, 7, 7, 7, 2, 7, 2, 5, 8, 8, 1, 8, 7, 7, 8, 8, 1, 1, 2, 1, 8, 8, 8, 7, 7, 7, 2, 5, 5, 8, 5, 5, 8, 5, 8, 1, 1, 7, 8, 5, 5, 5, 8, 7, 7, 2, 7, 2, 7, 7, 2, 1, 8, 7, 2, 5, 5, 8, 5, 8, 7, 7, 7, 8, 1, 8, 1, 1, 1, 5, 8, 8, 5, 1, 8, 8, 1, 2, 5, 5, 8, 5, 5, 1, 2, 5, 5, 5, 5, 5, 8, 5, 8, 8, 8, 8, 1, 7, 7, 2, 5, 8, 8, 8, 5, 8, 7, 2, 1, 8, 8, 8, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 1, 5, 5, 5, 8, 1, 1, 2, 2, 7, 7, 2, 5, 5, 8, 1, 8, 7, 1, 7, 7, 1, 8, 1, 2, 5, 8, 8, 1, 5, 5, 5, 8, 8, 1, 8, 1, 5, 5, 5, 5, 5, 8, 5, 5, 8, 8, 1, 2, 7, 7, 7, 2, 1, 8, 8, 5, 5, 1, 7, 2, 5, 5, 8, 5, 8, 8, 5, 1, 5, 8, 8, 7, 7, 2, 7, 7, 7, 2, 5, 5, 5, 5, 8, 1, 2, 7, 7, 2, 1, 8, 5, 8, 1, 8, 7, 2, 1, 7, 7, 7, 2, 5, 8, 8, 7, 1, 7, 7, 1, 8, 1, 1, 8, 8, 8, 5, 5, 8, 1, 1, 8, 7, 2, 5, 1, 8, 1, 2, 1, 1, 8, 1, 7, 1, 1, 8, 8, 8, 5, 8, 8, 8, 8, 7, 2, 7, 7, 7, 7, 7, 2, 7, 8, 7, 7, 2, 7, 1, 8, 5, 8, 1, 2, 5, 5, 8, 1, 2, 7, 1, 1, 7, 2, 7, 7, 2, 5, 5, 5, 8, 1, 5, 5, 5, 8, 8, 8, 5, 8, 1, 2, 2, 5, 8, 1, 8, 7, 7, 2,

2, 7, 7, 7, 7, 2, 2, 1, 5, 1, 8, 1, 1, 7, 8, 7, 7, 1, 2, 5, 5, 8, 1, 2, 1, 2, 1, 7, 7, 7, 2, 2, 5, 5, 1, 7, 7, 2, 5, 8, 7, 7, 8, 8, 7, 1, 7, 7, 7, 7, 1, 2, 7, 2, 2, 5, 8, 8, 7, 7, 1, 5, 8, 5, 5, 5, 1, 1, 7, 2, 5, 5, 5, 8, 8, 1, 2, 5, 8, 5, 5, 5, 5, 5, 8, 5, 8, 1, 1, 2, 2, 7, 2, 1, 1, 2, 7, 2, 5, 8, 8, 7, 8, 8, 5, 8, 5, 8, 8, 8, 7, 7, 7, 2, 1, 8, 8, 8, 5, 5, 1, 1, 2, 5, 5, 5, 5, 5, 5, 5, 8, 8, 1, 2, 7, 7, 7, 1, 2, 5, 8, 1, 2, 7, 8, 8, 7, 7, 7, 1, 2, 1, 2, 7, 7, 2, 5, 5, 5, 5, 5, 5, 8, 7, 2, 1, 5, 8, 7, 7, 2, 7, 7, 7, 7, 2, 2, 5, 8, 1, 7, 2, 5, 8, 1, 1, 8, 8, 8, 1, 7, 2, 7, 2, 2, 1, 5, 8, 7, 7, 1, 2, 2, 7, 8, 8, 7, 1, 2, 2, 5, 8, 1, 2, 7, 7, 1, 1, 8, 7, 2, 2, 5, 5, 8, 7, 8, 7, 1, 7, 1, 2, 2, 7, 7, 8, 8, 8, 5, 5, 5, 8, 7, 1, 8, 7, 7, 8, 5, 5, 8, 5, 1, 8, 1, 8, 7, 7, 8, 7, 7, 7, 7, 2, 1, 8, 5, 5, 5, 5, 8, 8, 1, 1, 7, 7, 7, 2, 7, 8, 7, 2, 5, 5, 8, 8, 7, 7, 7, 7, 2, 5, 5, 5, 1, 8, 1, 7, 7, 7, 2, 7, 8, 5, 8, 1, 8, 7, 1, 1, 5, 8, 5, 5, 5, 8, 7, 7, 7, 2, 5, 8, 1, 2, 2, 5, 1, 8, 7, 2, 7, 1, 1, 2, 5, 8, 8, 8, 1, 8, 7, 2, 5, 8, 8, 7]

По траекториям видно что не происходит выхода из класса $\mathbf{C}_1$. Из несущественных состояний 3 и 6 цепь уходит в класс $\mathbf{C}_1$ и больше в $\mathbf{C}_2$ не возвращается.

№ 7:

В задании 2 цепь была разбита на классы. Тут рассматривается замкнутый класс $\mathbf{C}_1 = \{1, 2, 5, 7, 8\}$. Так как класс непериодичен то для него была взята матрица переходов $\mathbb{P}_{\mathbf{C}_1}$ из задания 5. Траектории внутри класса моделируются функцией simulate_markov_chain из задания 6.

Функция calculate_state_percentages считает долю времени проведенного цепью в каждом состоянии, как число попаданий в состояние, деленное на длину траектории.

```
def calculate_state_percentages(states, num_states):
    counts = np.bincount(states, minlength=num_states)
    percentages = counts / len(states)
    return percentages
```

Листинг 2: Функция подсчета доли времени в каждом состоянии

Функция stationary_distribution находит вектор финальных вероятностей как собственный вектор матрицы $\mathbb{P}^T$ для собственного значения 1, нормированный так, чтобы сумма была равна 1.

```
def stationary_distribution(transition_matrix):
    eigenvalues, eigenvectors = np.linalg.eig(transition_matrix.T)
    stationary_vector = eigenvectors[:, np.isclose(eigenvalues, 1)]
    stationary_vector = stationary_vector / np.sum(stationary_vector)
    return stationary_vector.real.flatten()
```

Листинг 3: Функция нахождения вектора финальных вероятностей

Функция simulation моделирует траекторию заданной длины из заданного состояния и печатает доли времени, вектор финальных вероятностей и модуль их разности.

```
def simulation(steps, initial_state, transition_matrix, labels):
    print(f"Initial state: {labels[initial_state]}, steps: {steps}")

    trajectory = simulate_markov_chain(transition_matrix, initial_state
        , steps)
```

```
    percentages = calculate_state_percentages(trajectory, len(
        transition_matrix))
    print(f"Percentages: {percentages}")
    stationary_vector = stationary_distribution(transition_matrix)
    print(f"Final distribution: {stationary_vector}")

    difference = np.abs(percentages - stationary_vector)
    print(f"Difference: {difference}\n")
```

Листинг 4: Функция моделирования и сравнения с финальными вероятностями

Результаты (Initial state - начальное состояние, steps - число шагов, Percentages - доля времени в каждом состоянии класса, Final distribution - вектор финальных вероятностей класса, Difference - модуль разности этих векторов):

```
Class 1: states (1, 2, 5, 7, 8)

Initial state: 1, steps: 10
Percentages: [0.3636 0.2727 0.1818 0.0909 0.0909]
Final distribution: [0.1837 0.1573 0.2146 0.2357 0.2088]
Difference: [0.1799 0.1155 0.0328 0.1448 0.1179]

Initial state: 2, steps: 50
Percentages: [0.1765 0.2157 0.1961 0.3137 0.098 ]
Final distribution: [0.1837 0.1573 0.2146 0.2357 0.2088]
Difference: [0.0072 0.0584 0.0185 0.0781 0.1107]

Initial state: 5, steps: 100
Percentages: [0.2079 0.0495 0.2178 0.2772 0.2475]
Final distribution: [0.1837 0.1573 0.2146 0.2357 0.2088]
Difference: [0.0242 0.1078 0.0032 0.0416 0.0387]

Initial state: 7, steps: 1000
Percentages: [0.1908 0.1828 0.2038 0.2288 0.1938]
Final distribution: [0.1837 0.1573 0.2146 0.2357 0.2088]
Difference: [0.0071 0.0256 0.0108 0.0069 0.015 ]
```

Получается чем длиннее траектория - тем ближе вектор процентов времени нахождения к вектору финальных вероятностей.

Вектора финальных вероятностей совпали с посчитанными в пункте 5.